OpenTelemetry在企业内部应用所需要的技术栈

Original crossoverJie crossoverJie 2024年09月17日 19:57

crossoverJie
技术、生活、观点、原创。 原创公众号； 主要关注 Go、JVM、并发、分布式、网络...
219篇原创内容

公众号

可观测性概念



当一个软件或系统出于运行状态时，如果我们不对他加以观测，那它的运行状态对我们来说就是一个黑盒。

> 如上图所示。

我们只能通过业务的表象来判断它是否正常运行，无法在故障发生前进行预判，从而只能被动解决问题。

这类问题在微服务时代体现的更加明显，即便是业务已经出现问题，在没有可观测性系统的前提下想要定位问题更是难上加难。



好在可观测性这个概念由来已久，已经由一些业界大佬抽象出几个基本概念：

- Logs：离散的日志信息
- Metrics：聚合的指标
- Trace：请求基本的链路追踪

结合这三个指标，我们排查问题的流程一般如下：

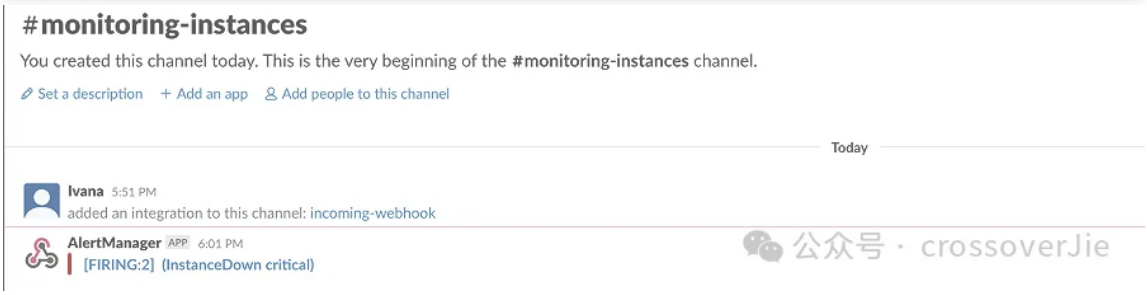


首先根据 metrics 来判断是否有异常，这点可以通过在 Prometheus 的 AlertManager 配置一些核心的告警指标。

比如当 CPU、内存使用率超过 80% 或者某个应用 Down 机后就发出告警。

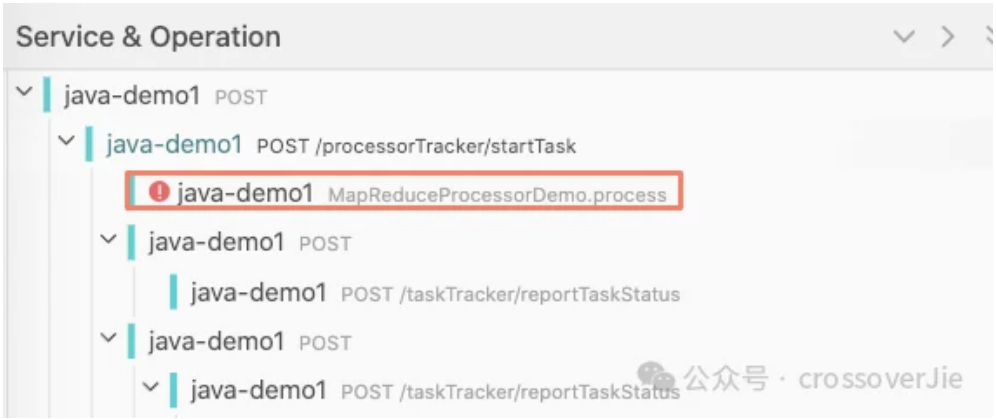
```
groups:
- name: AllInstances
  rules:
  - alert: InstanceDown
    # Condition for alerting
    expr: up == 0
```

```
    for: 1m
    # Annotation - additional informational labels to store more information
    annotations:
      title: 'Instance {{ $labels.instance }} down'
      description: '{{ $labels.instance }} of job {{ $labels.job }} has been down for mor
    # Labels - additional labels to be attached to the alert
    labels:
      severity: 'critical'
```



这可以让我们尽早发现故障。

之后我们可以通过链路信息找到发生故障的节点。



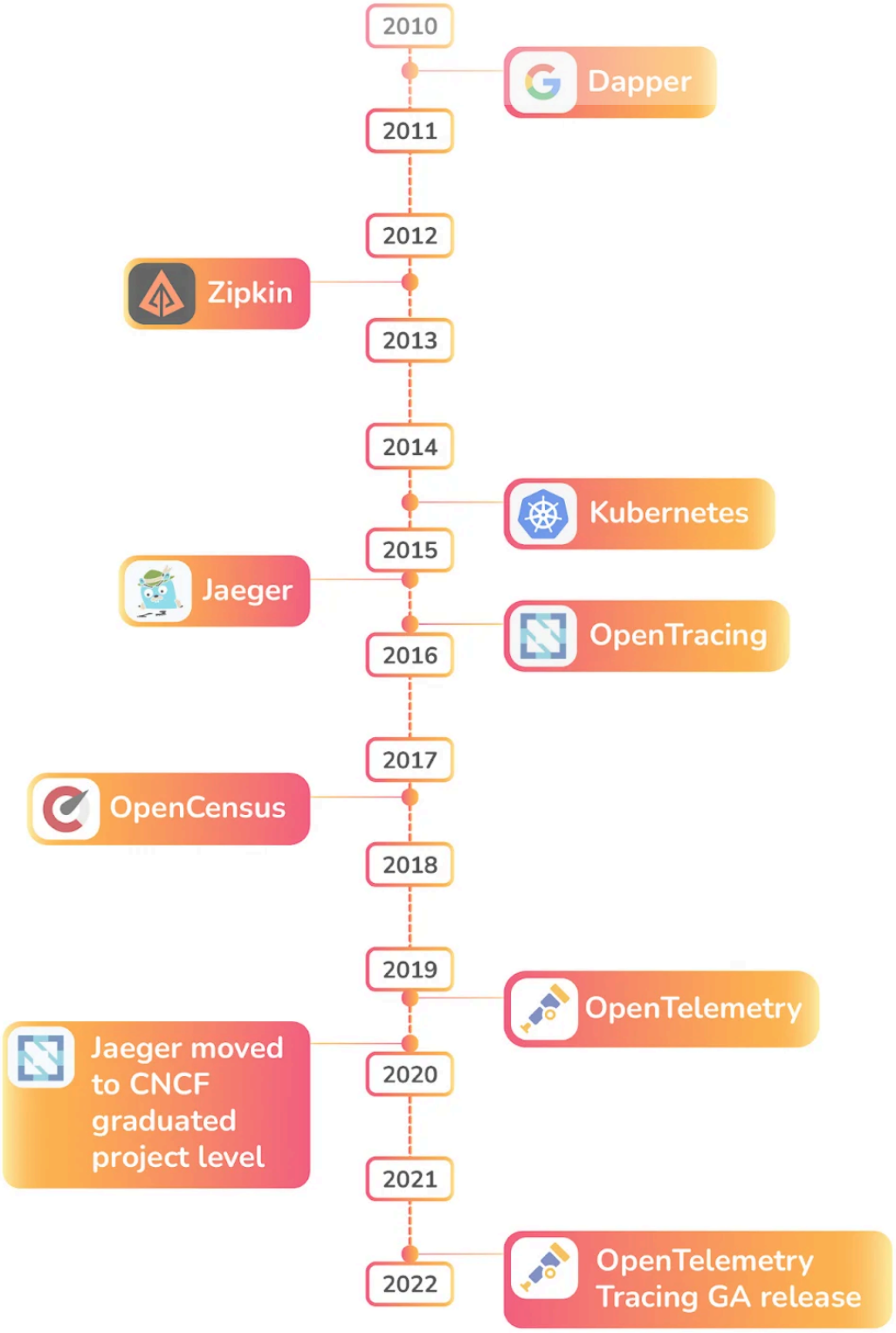
然后通过这里的 trace_id 在应用中找到具体的日志：

```
mdc.trace_id:4a686dedcdf4e95b1a83b36e62563a96
```

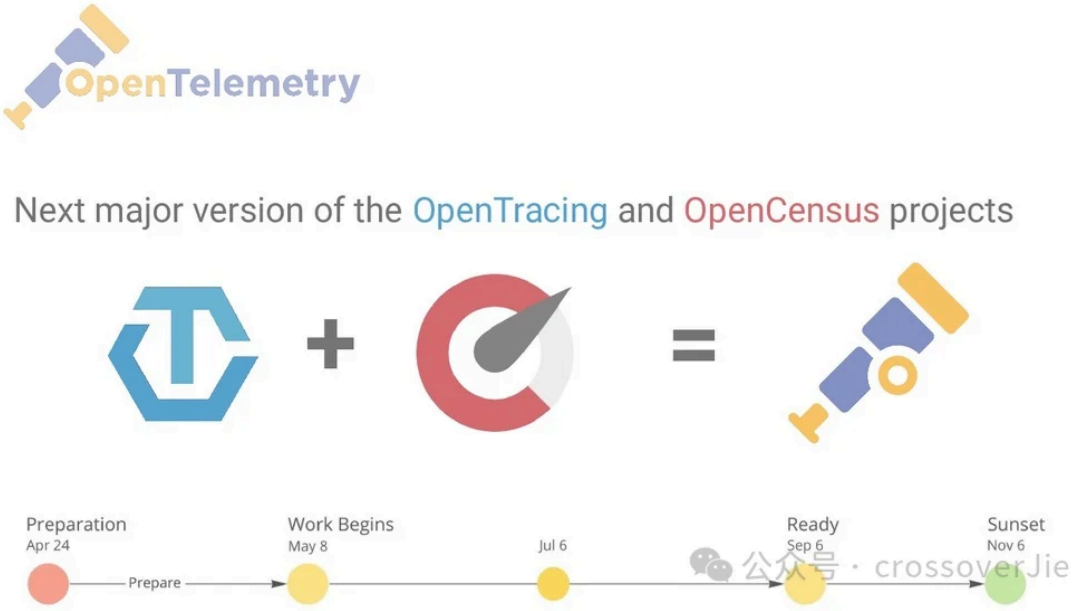
再根据日志中的上下文确定具体的异常原因。

这就是一个完整的排查问题的流程。

OpenTelemetry 发展历史



公众号 · crossoverJie



在 OpenTelemetry 开始之前还是先回顾下可观测性的发展历史，其中有几个重要时间点：

- 2010 年 Google 发布了 Dapper 论文，给业界带来了实现分布式追踪的理论支持，之后的许多分布式链路追踪实现都有它的影子
- kubernetes 的发布奠定了后续云原生社区的基础
- Jaeger 发布后成为了主流的链路存储系统
- 2019 年 OpenTracing 和 OpenCensus 合并为 OpenTelemetry
- 2021 年底 OpenTelemetry 发布第一个 GA release 版本

OpenTelemetry 是什么?



以前我们所接触到的类似于阿里的ARMS、美团的 CAT、Pinpoint 这类系统大多都有一个公司在背后进行驱动，与厂商绑定的非常紧密。

而 OpenTelemetry 则相反，它主要由社区驱动，参与的公司众多；同时它定义和提供了一套可观测性的标准（包括 API、SDK、规范等数据）。

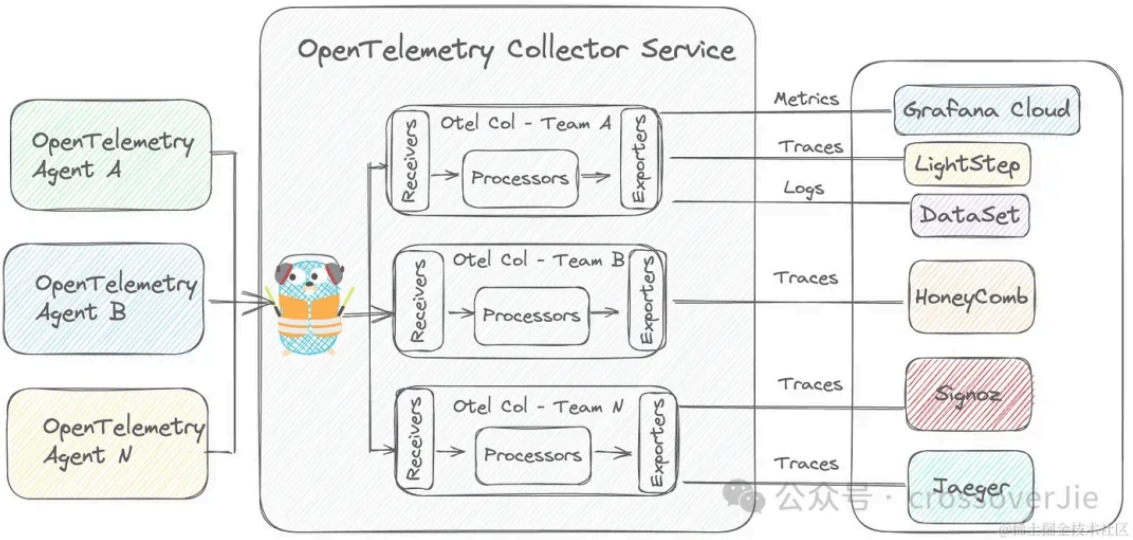
使用它你可以灵活的选择和搭配任意的开源或商业产品来组成你的可观测性技术栈。

Language	Traces	Metrics	Logs
C++	Stable	Stable	Stable
C#/.NET	Stable	Stable	Stable
Erlang/Elixir	Stable	Development	Development
Go	Stable	Stable	Beta
Java	Stable	Stable	Stable
JavaScript	Stable	Stable	Development
PHP	Stable	Stable	Stable
Python	Stable	Stable	Development
Ruby	Stable	Development	Development
Rust	Beta	Alpha	Alpha
Swift	Stable	Development	Development

公众号 · crossoverJie

因为社区非常活跃，所以当前也几乎支持主流的开发语言。

OpenTelemetry 的架构

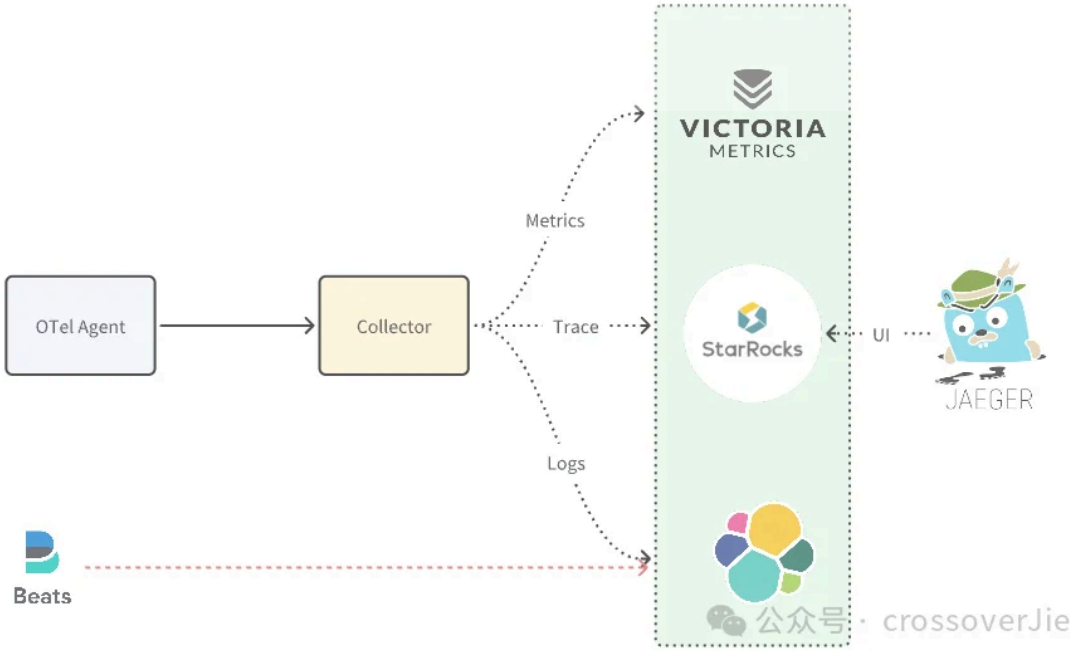


公众号 · crossoverJie
@稀土掘金技术社区

OpenTelemetry 的架构主要分为三个部分：

- 左侧的客户端 Agent，用于采集客户端的数据，通常就是我们的应用。
- 中间的是 Collector-Service，用于接受客户端的数据、内部处理、导出数据到各种存储
- 右侧的则是各种存储层，用于存储 Metrics、Logs、Traces 这些数据。

我们基于官方推荐的技术架构选型了我们的技术栈：



主要的区别就是使用 VictoriaMetrics 存储指标、StackRocks 存储 Trace，ElasticSearch 存储日志。

> 只是目前我们的日志链路还没有完全切换到 OpenTelemetry 的链路，依然是在 Pod 中挂载了一个 sidecar，在这个 sidecar 中通过 filebeat 采集日志输出到 elasticsearch，后续也会逐步迁移。

核心项目

Collecotor

OpenTelemetry 社区的项目众多，其中大部分都是各种语言的 SDK 和 API，其中最为关键的应该就是 opentelemetry-collector

也就是刚才架构图中的中间部分，我们可以把它理解为类似 APIGateway 的角色，所有上报的 OTel 数据都得经过它的处理。



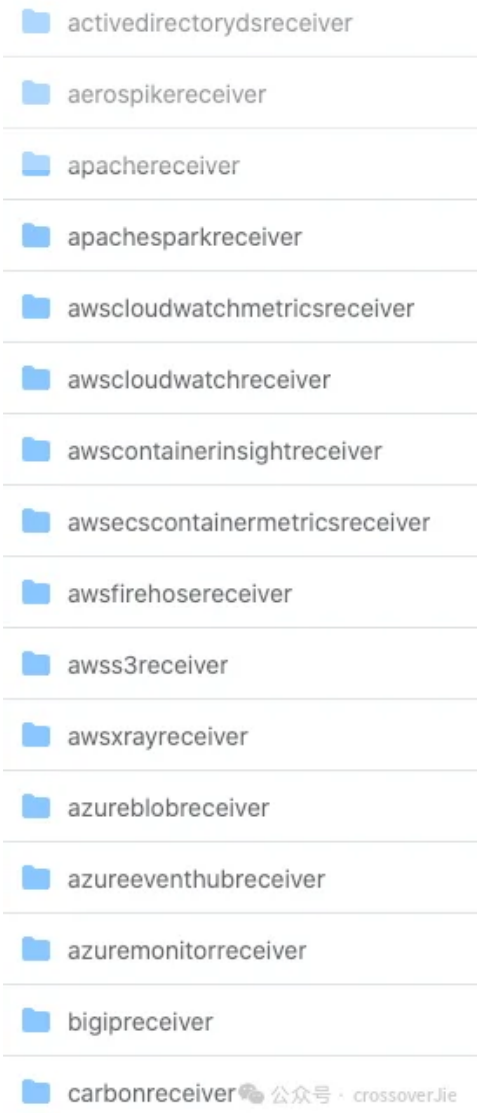
OTEL COLLECTOR

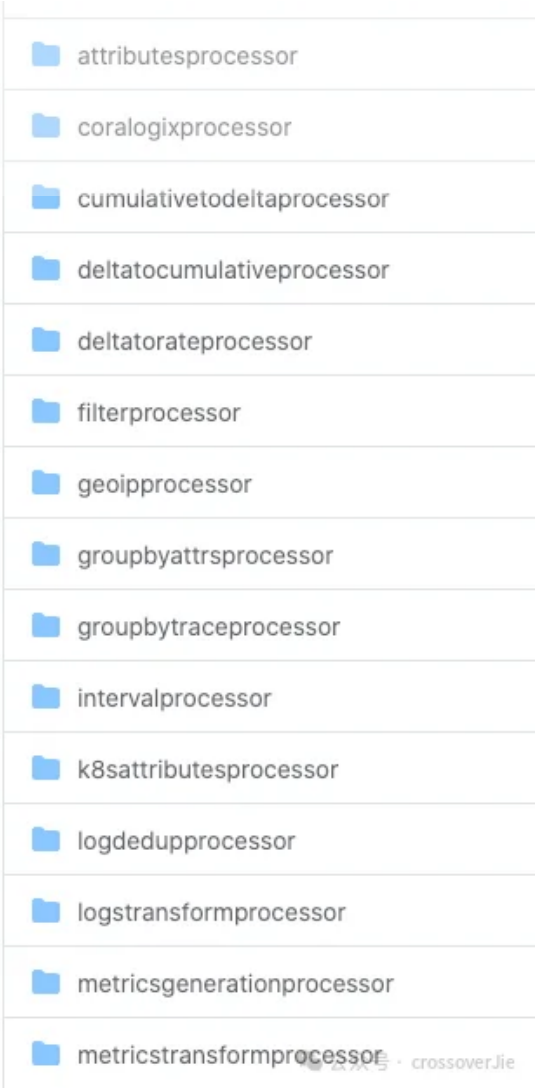
公众号 · crossoverJie
@稀土掘金技术社区

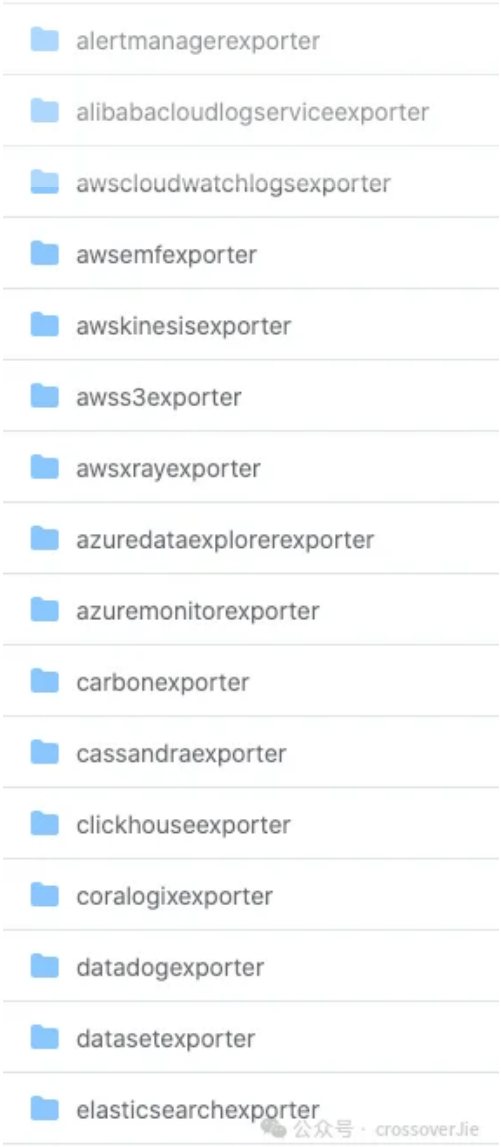
主要由以下三部分组成：

- Receiver：用于接受客户端上报的数据
- Process：内部的数据处理器
- Exporter：将数据导出到不同的存储

由于 OpenTelemetry 社区非常的活跃，所以这里支持的 Receiver、Processor 和 Exporter 类型非常多。


activedirectorydsreceiver
aerospikereceiver
apachereceiver
apachesparkreceiver
awscloudwatchmetricsreceiver
awscloudwatchreceiver
awscontainerinsightreceiver
awsecscontainermetricsreceiver
awsfirehosereceiver
awss3receiver
awsxrayreceiver
azureblobreceiver
azureeventhubreceiver
azuremonitorreceiver
bigipreceiver
carbonreceiver
公众号 · crossoverJie


attributesprocessor
coralogixprocessor
cumulativetodeltaprocessor
deltatocumulativeprocessor
deltatorateprocessor
filterprocessor
geoipprocessor
groupbyattrsprocessor
groupbytraceprocessor
intervalprocessor
k8sattributesprocessor
logdedupprocessor
logstransformprocessor
metricsgenerationprocessor
metricstransformprocessor
crossoverJie



其他核心项目

我们以 Java 为例，对业务开发最重要的库就是 opentelemetry-java-instrumentation

它可以打包一个 javaagent 给我们使用：

```
# Java example
java -javaagent:path/to/opentelemetry-javaagent.jar \
     -jar myapp.jar
```

Libraries / Frameworks

These are the supported libraries and frameworks:

Library/Framework	Auto-instrumented versions	Standalone Library Instrumentation [1]	Semantic Conventions
Akka Actors	2.3+	N/A	Context propagation
Akka HTTP	10.0+	N/A	HTTP Client Spans, HTTP Client Metrics, HTTP Server Spans, HTTP Server Metrics, Provides `http.route` [2]
Alibaba Druid	1.0+	opentelemetry-alibaba-druid-1.0	Database Pool Metrics
Apache Axis2	1.6+	N/A	Provides `http.route` [2], Controller Spans [3]
Apache Camel	2.20+ (not including 3.x yet)	N/A	Dependent on components in use
Apache CXF JAX-RS	3.2+	N/A	Provides `http.route` [2], Controller Spans [3]
Apache CXF JAX-WS	3.0+	N/A	Provides `http.route` [2], Controller Spans [3]
Apache DBCP	2.0+	opentelemetry-apache-dbcp-2.0	Database Pool Metrics
Apache Dubbo	2.7+	opentelemetry-apache-dubbo-2.7	RPC Client Spans, RPC Server Spans
Apache HttpAsyncClient	4.1+	N/A	HTTP Client Spans, HTTP Client Metrics
Apache HttpClient	2.0+	opentelemetry-apache-httpclient-4.3, opentelemetry-apache-httpclient-5.2	HTTP Client Spans, HTTP Client Metrics

公众号 · crossoverJie

同时也支持了我们日常开发的绝大多数框架和中间件。

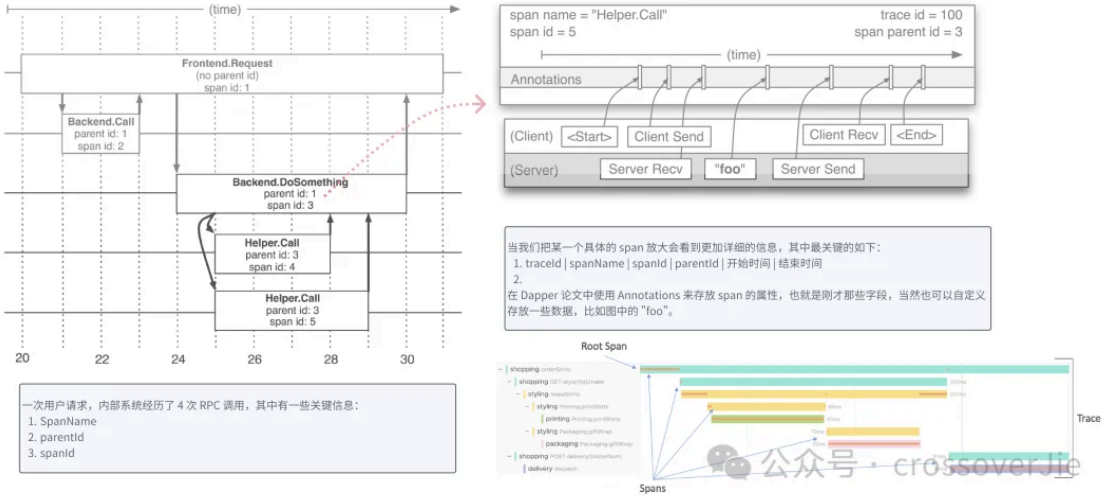
> 支持的库与框架列表

如果我们需要在应用中自定义打桩一些 Span、Metrics ，就还需要 opentelemetry-java 这个项目。

它提供了具体的 SDK 可以方便的创建 Span 和 Metrics。

Trace

之后来看看 `OpenTelemetry` 中具体的三个维度的概念和应用，首先是 Trace。



Trace 这个概念首先是 Google Dapper 论文中提到。

如上图所示：一次用户请求经历了 4 次 PRC 调用，分别也属于不同的系统。

每一次 RPC 调用就会产生一个 Span，将这些 span 串联起来就能形成一个调用链路。

这个 Span 主要包含以下信息：

- SpanName
- ParentID
- SpanID

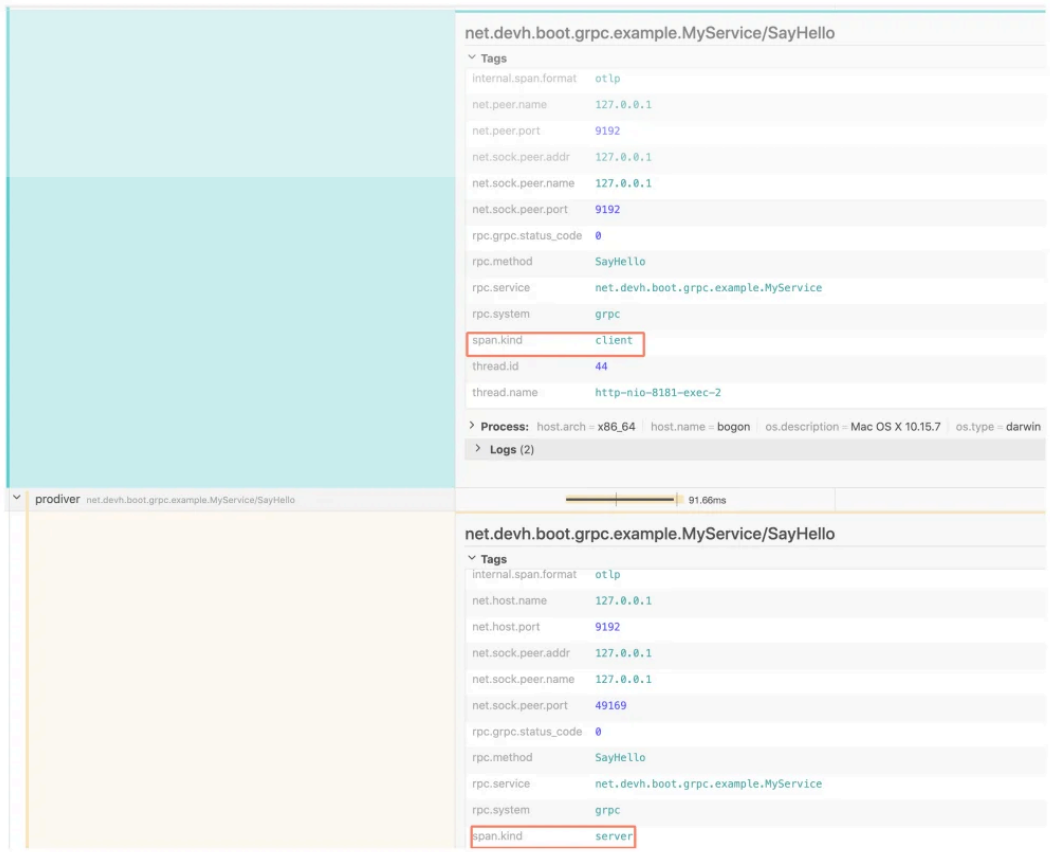
当我们将一个 Span 放大后会看到更加具体的信息：

- TraceId
- SpanName
- ParentID
- SpanID
- 开始时间
- 结束时间 在 Dapper 论文中使用 Annotations 来存放 span 的属性，当然也可以自定义存放一些数据，比如图中的 `"foo"` 。

> 在 OpenTelemetry 的 SDK 中称为 attribute，而在 Jaeger 的 UI 中又称为 tag，虽然叫法不同，但本质上是一个东西。

最终就会形成上图中的树状结构的调用关系。

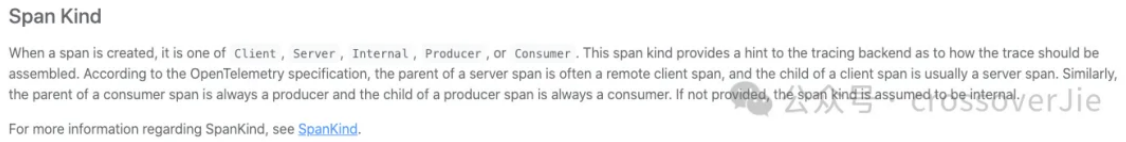
Span Kind



When a span is created, it is one of Client, Server, Internal, Producer, or Consumer.

公众号 · crossoverJie

Span 中还有一个非常重要的概念，就是 Span Kind，也就是 Span 的类型，这个类型可以在排查问题时很容易得知该服务的类型。



公众号 · crossoverJie

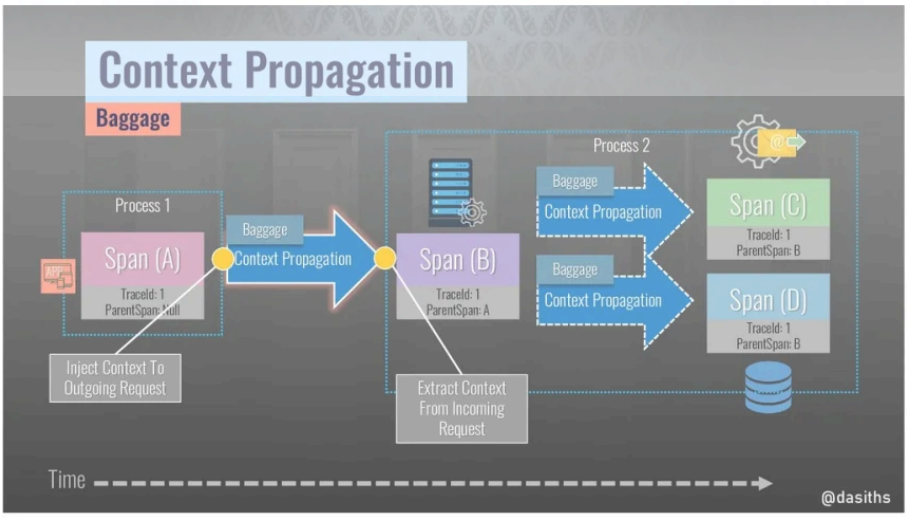
按照官方的定义，Span 的类型分为：

- Client
- Server
- Internal
- Producer
- Consumer

对于 RPC 的客户端和服务端自然就对应 Client 和 Server，而使用了消息队列的生产者消费者对应的就是 Produce 和 Consumer。

除此之外发生在应用内部的一些关键 Span 的类型就是 Internal，比如我们需要对业务的某些关键函数生成 Span 时，此时的 Span 类型通常也都是 Internal。

上下文传递



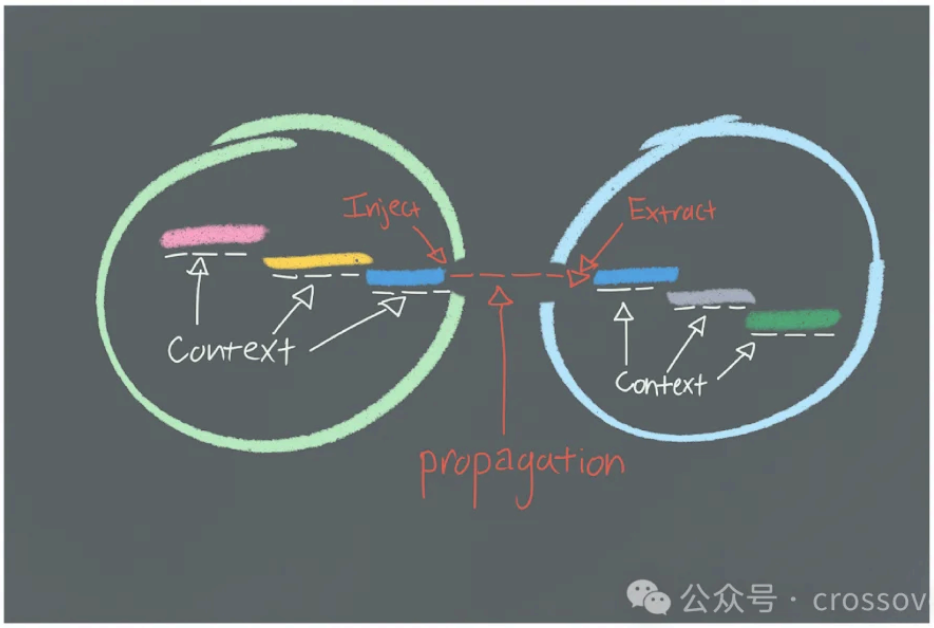


image.png

在 Trace 中有一个关键技术问题需要被解决，也就是 Context 的上下文传递。

这个特别是在分布式系统中必须要解决，我们可以简单把它理解为如何把上游生成的 trace_id 传递到下游，这样才能在追踪的链路追踪系统中串联起来。

这个关键的技术名词在 OpenTelemetry 中称为：Context Propagation.

在分布式系统中，数据都是通过网络传递的，所以这里的本质问题依然是如何将上下文数据序列化之后，在下游可以反序列化到 `Context` 中。

聪明的小伙伴应该已经想到，我们可以将 trace_id 写入到跨进程调用的元数据中：

- http 可以存放在 http header 中
- gRPC 可以存放在 meta 中

- Pulsar 可以存放在消息的 properties 中
- 其余的中间件和框架也是同理

然后在远程调用之前使用 `Inject` 将数据注入到这些元数据里，下游在接收到请求后再通过一个 `Extract` 函数将元数据解析到 `Context` 中，这样 `trace_id` 就可以串联起来了。



image.png

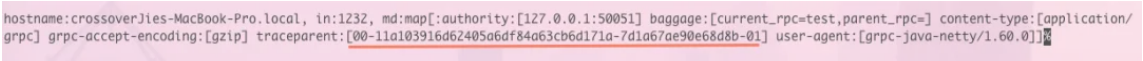
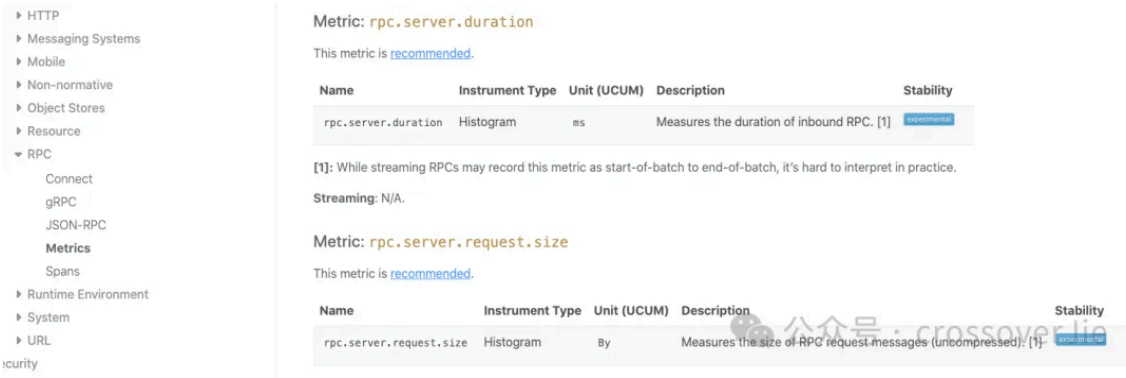


image.png

上图就是 Pulsar 和 gRPC 传递 trace_id 的过程，数据都是存放在元数据中的，这里的 `traceparent` 的值本质上就是 trace_id.

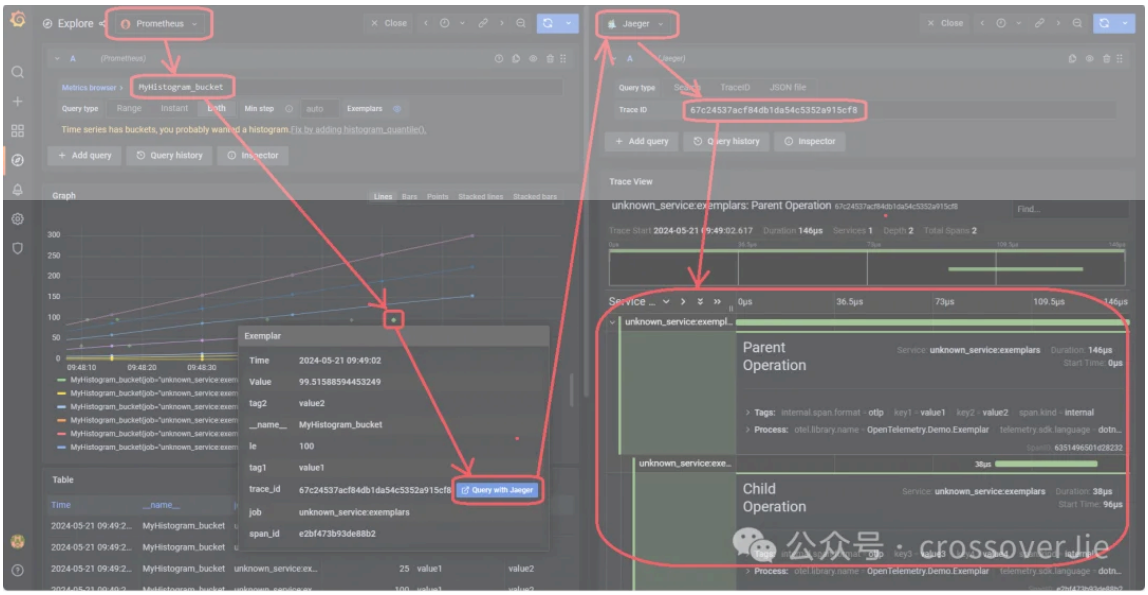
> 具体的代码细节我会在下一篇继续分析。

Metrics



Metrics 相对于 Trace 来说则是要简单许多，OpenTelemetry 定义了许多命名规范和标准，这样大家在复用社区的一些监控模板时就要更加容易一些。

Metrics Exemplars



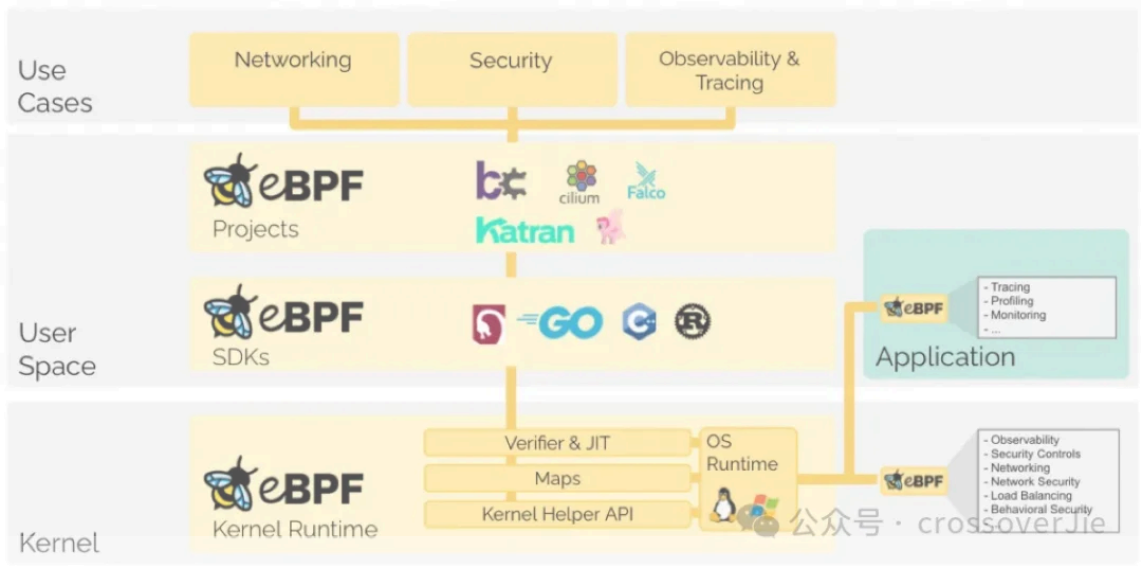
Metrics 还提供了一个 Exemplar 的功能，它的主要作用是可以将 Metrics 和 Trace 关联在一起，这样在通过 Metrics 发现问题时，就可以直接跳转到链路系统。

因为 trace_id 可以通过 MDC 和日志关联，所以我们可以直接通过 Metrics 定位具体应用的日志，这样排查问题的效率将会非常高。

扩展信息

以上就是关于 OpenTelemetry 的整体架构，下面来扩展一些内容。

eBPF



eBPF 是一个运行在 Linux 内核中的虚拟机，它提供一套特殊的指令集并允许我们在不重新编译内核、也不需要重启应用的情况下加载自定义的逻辑。

eBPF 技术具有三大特点：

- 第一是**无侵入**，动态挂载，目标进程无需重启，而且因为是 Linux 内核提供功能，所以与语言无关，任何语言都可以支持。
- 第二是**高性能**，eBPF 字节码会被 JIT 成机器码后执行，效率非常高；
- 第三是更加**安全**，它会运行在自己的沙箱环境中，不会导致目标进程崩溃。

eBPF 虽然有很多优点，同时也有一些局限性，比如我想监控业务代码中的某个具体指标（订单创建数量），此时它就难以实现了，所以还得看我们的应用场景。更适合一些云平台，或者更偏向底层的应用。

目前 eBPF 的应用场景还不够广泛，但假以时日一定会成为可观测领域的未来之星。

SigNoz

不知道大家发现没有，如果我们直接 OpenTelemetry 技术栈会需要为 Trace、Metrics、Logs 选择不同的存储，而且他们的查询界面也分散在不同的地方。

那有没有一个统一的平台可以给我们提供完整的可观测体验呢?

有这样的需求那就有对应的厂商实现了：

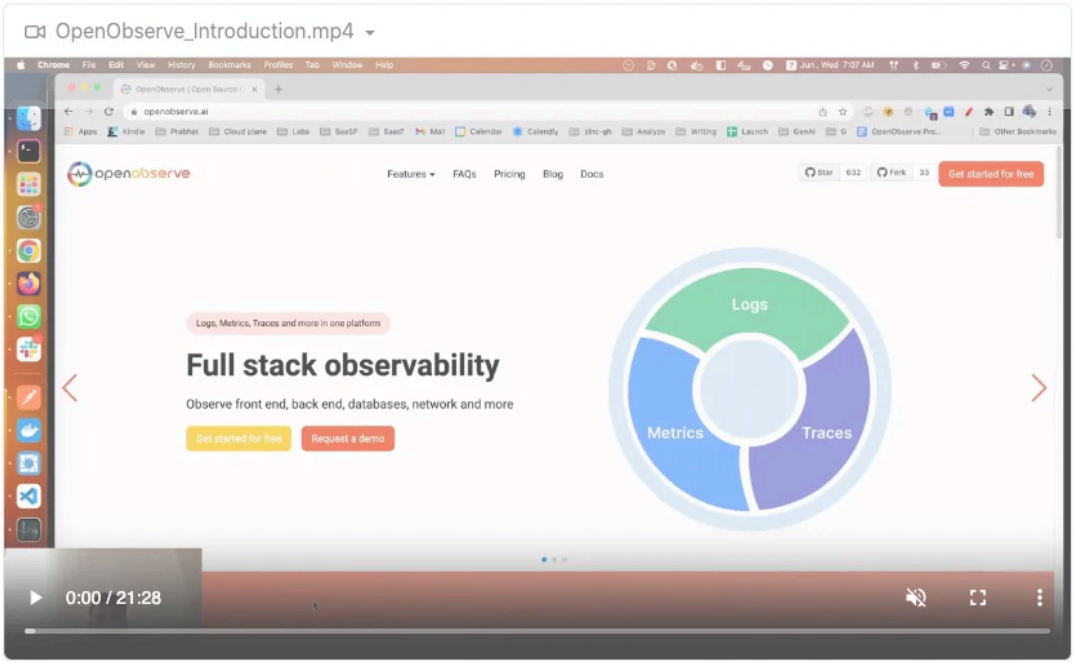


SigNoz 就是这样的平台，它将 OpenTelemetry-collector 和数据存储全部整合在了一起，同时全面兼容 OpenTelemetry；可以说它就是基于 OpenTelemetry 构建的一个可观测产品。

对于一些中小厂商，不想单独维护这些组件时是非常有用的。

OpenObserve

Introduction Video



Features:

- **Logs, Metrics, Traces**: Comprehensive support for various data types.
- **OpenTelemetry Support**: Full compatibility with OTLP for logs, metrics, and traces.
- **Real User Monitoring (RUM)**: Includes performance tracking, error logging, and session replay.
- **Alerts & Dashboards**: Features over 14 different chart types for comprehensive data visualization.
- **Advanced Ingest and Query Functions**: Aid in enrichment, redaction, log reduction, and compliance, like redacting sensitive data from logs.
- **Advanced Embedded GUI**: Intuitive and user-friendly interface.
- **SQL and PromQL Support**: Query logs and traces with SQL, and metrics with SQL and PromQL.

公众号 · crossoverJie

image.png

OpenObserve在 SigNoz 的基础上做的更加极致一些，它提供了一个统一的存储可以存放日志、Trace、Metrics 等数据。

这样我们就可以只使用一个数据库存放所有的数据，同时它也提供了完整的 UI，并且也全面兼容 OpenTelemetry。

这样对于运维来说会更加简单，只是可能带来的副作用就是需要与它完全绑定。

总结

以上就是 OpenTelemetry 在企业的应用，大家可以根据自己的情况选择自建 OTel 的技术栈，还是选择 SigNoz 和 OpenObserve 这类的标准化产品。

最后夹带一点私活，最近做了一个知识星球，已经试运行一段时间，效果还不错（具体详情可以扫码查看）；感兴趣的朋友可以扫码领取优惠券以 49 元的价格加入（支持三天内无条件退款）。



往期推荐

日志与追踪的完美融合：OpenTelemetry MDC实践指南

OpenTelemetry 实战：gRPC 监控的实现原理

OpenTelemetry 实战：从零实现应用指标监控

OpenTelemetry 实战：从零实现分布式链路追踪

深入理解单元测试：技巧与最佳实践

点分享 点收藏 点点赞 点在看

OpenTelemetry 14

OpenTelemetry · 目录

上一篇 · 日志与追踪的完美融合：OpenTelemetry MDC实践指南

Read more